HANDOFF PLAYBOOK · STACK-AGNOSTIC

GIFTMYBOOK · KNOWLEDGE TRANSFER

Same character, *every* image.

A playbook for keeping characters and objects consistent across a set of independently generated AI images. Written for a team whose stack and product we do not know.



FOR
A product, engineering and ML team building multi-image generation

ASSUMES
No knowledge of our code, our stack or our product

STATUS
Version 1 · November 2026

01

START HERE

What this document is, and how to use it

We spent about twenty engine revisions learning how to make an image model draw the same child, in the same clothes, with the same toy, twelve times in a row. This playbook hands you the method.

It does not hand you our code. We do not know your stack, your image model or your product, so everything here is written as roles, contracts and patterns you can implement in any language against any model that meets a short list of capabilities (chapter 04).

TRANSFERS AS IS

The method

The five moves, the component roles, the data contracts, the prompt patterns, the blocking / advisory split, the acceptance algorithm and the pitfalls.

TRANSFERS AS A STARTING VALUE

The numbers

Every threshold and budget we quote was calibrated on our model, our art style and our subject. Treat each as a first guess to re-measure on your own data.

DOES NOT TRANSFER

The specifics

Our model choice, storage layout, book format, catalogue of stories and the rules that exist only because our subject is a child in a picture book.

Reading paths

IF YOU ARE	READ	YOU WILL LEAVE WITH
Deciding whether to adopt this	02, 03, 09	The idea, what we need to know about you, and what it costs to build
Designing the system	03, 04, 05, 07	A component diagram, the contracts between components, and the acceptance algorithm
Writing prompts and judges	06, 07, 10	Prompt patterns, the defect ledger, and the mistakes to avoid
Running the project	09, 11	A phased plan with exit criteria, and the handoff process we propose

VOCABULARY USED THROUGHOUT

Set: the group of images that must agree (for us, the twelve spreads of one book). **Item**: one image in the set. **Subject**: the main character. **Identity kit**: the fixed references built once per set. **Judge**: a vision model asked to check an image. **Candidate**: one generated attempt at an item.

THE IDEA

02 Why sets drift, and the five moves that stop it

An image model called twelve times has no memory of the other eleven calls. Whatever you do not pin, it reinvents. We found four causes, and none of them is specific to children's books.

CAUSE 1

The fixed inputs were words

"A red jacket" leaves the cut, the length, the collar and the shade free. Image models weight pictures over prose.

CAUSE 2

The reference was the wrong kind of image

A single hero image is a scene. It crops, poses and lights. Whatever it does not show, usually legs and shoes, has no pixels behind it.

CAUSE 3

Checks were lenient yes / no questions

"Does the outfit match?" asked of one image against a sentence passes almost anything. Nothing compared pixels with pixels.

CAUSE 4

Repair was a blind re-roll

Regenerating with the same inputs and hoping. The result was then cached and replayed for ever.

The five moves

1. **Pin every identity as pixels plus a spec, once per set**
   For each thing that must stay the same, build a purpose-made reference image (a model sheet, not a scene) and a short structured description read off that image. Build them before generating any item. Never rebuild them mid-set.
2. **Attach the same labelled reference pack to every item**
   Same images, same order, same labels. Each label says what the image is for and what must not be copied from it.
3. **Verify each item against the kit, structurally**
   Show the judge the item *and* the references. Ask for a schema: per garment, per object, per character. Map each answer to a fixed defect string, classified in advance as blocking or advisory.
4. **Rank candidates and repair under a budget**
   Blocking defects trigger a repair render steered by a fixed note built from the kit. Cap the total renders per item. Never adopt a result you could not verify.
5. **Judge the finished set as a set**
   Crop the subject from every item, tile the crops beside the model sheet in one image, and ask which ones differ. Re-render the outliers.

THE ONE RULE UNDERNEATH ALL FIVE

A stateless generation can only be held to fixed inputs, and an invariant that no check enforces is an aspiration. If something must be the same in every image, there must be one pinned input that says so and one check that fails when it is not.

03

DISCOVERY

What we need to know about you first

The method bends in predictable ways depending on your product and your model. Answering these before the first working session will save both teams a week. A blank column is left for your notes.

#	QUESTION	WHY IT CHANGES THE DESIGN	YOUR ANSWER
1	What is a "set" in your product, and how many images are in one?	Sets the cost of the set-level check and whether one contact sheet is enough	
2	Who or what must stay the same? One subject, several, objects, a setting?	Each one needs its own reference and its own checks	
3	Where does the subject's identity come from: a photo of a real person, a design you own, a text description?	Decides what the "ground truth" is and which privacy and safety limits apply	
4	Does a person approve anything before the set is generated?	An approved image is the best possible identity anchor	
5	Can your image model take several reference images in one call? How many?	The reference pack depends on it. See the fallback table below	
6	Can you fine-tune or train an adapter per subject?	Can replace the model sheet for identity, but not the checks	
7	Do you have a vision model that returns structured output reliably?	The whole verification layer runs on it	
8	Is the output printed, shown on screen, or animated?	Print makes defects permanent and justifies stricter gates	
9	What is the acceptable time and cost per set?	Sets candidate counts, repair budgets and which gates you can afford	
10	What happens today when a set is inconsistent? Who notices?	Tells us whether to stop and ask a human or ship with a flag	
11	Does the text or script behind each image name the objects and characters in it?	Needed to plan which object appears where	
12	Do you already store generated images with stable keys you can reuse?	Caching and resume depend on content-addressed keys	

How the answers change the design

IF	THEN
The model takes only one reference image	Composite the kit into a single reference board (subject sheet plus the objects for this item), and lean harder on the structured text spec and on verification.
The model takes no reference images	Identity must come from a fine-tune or adapter. Keep everything from move 3 onward unchanged; verification matters more, not less.
You fine-tune per subject	The fine-tune carries the face. You still need the outfit spec, the object references and every check, because fine-tunes drift on clothing and props.

Identity comes from a real person's photo	Let a person approve one generated image first, and anchor on that, not on the photo. Use the photo only as an advisory signal for the judge.
Sets are larger than about twelve images	Run the set check in windows that each include the model sheet, and add an embedding-based outlier test to pre-filter.
There are several recurring characters	One sheet and one spec per character, a rule for which characters are in which image, and a duplication check per character.
There is no human in the loop and output is permanent	Prefer stopping on blocking defects with the candidates attached over shipping with a flag.

ARCHITECTURE

04 Eight components, described by role

None of this assumes a language, a cloud or a model vendor. Each box is a role with an input and an output. Two are thin adapters around models you already have.

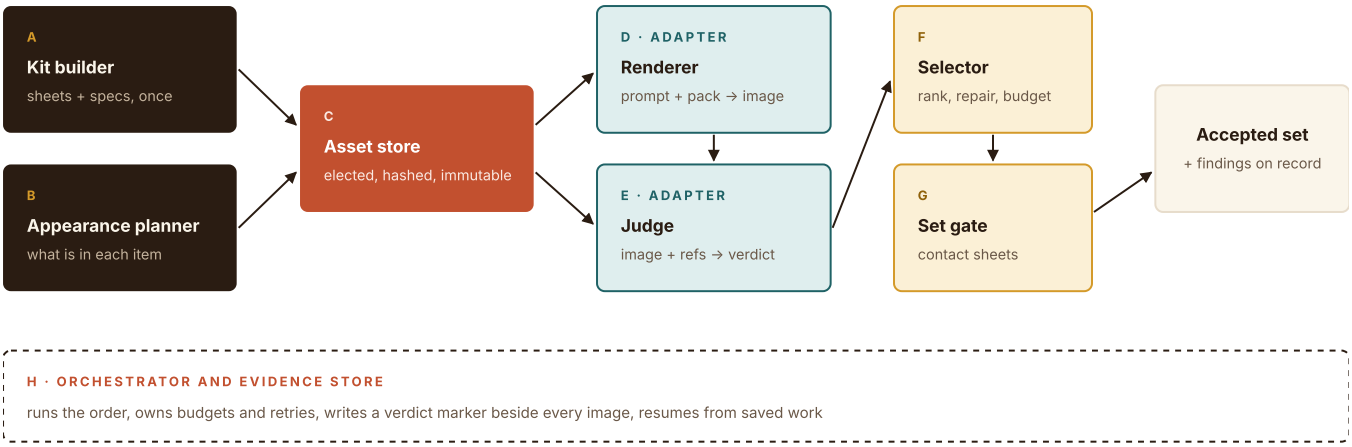


FIG. 1 The reference architecture. Only D and E touch a model vendor; swap them without changing the rest.

	COMPONENT	RESPONSIBILITY	CONTRACT
A	Kit builder	Builds a model sheet and a spec for each subject, object and recurring character. Generates candidates, judges them, elects one.	In: the identity anchor. Out: image + spec + content hash per reference.
B	Appearance planner	Decides, per item, which characters and objects are in the picture and in what state.	In: the script per item. Out: a schedule with visibility and state.
C	Asset store	Stores each reference once. First writer wins; everyone else adopts the winner. Never overwrites.	Create-if-absent by key; read by key; a manifest with one hash over all of it.
D	Renderer adapter	Turns a scene, the prompt blocks and the reference pack into one image. Hides vendor quirks and safety fallbacks.	In: scene, blocks, ordered labelled references. Out: image or a typed failure.

E	Judge adapter	Asks a vision model a schema-shaped question about one or more images.	In: images, question, schema. Out: a validated verdict or "unavailable".
F	Selector	Ranks candidates, decides whether to repair, enforces the per-item budget.	In: candidates with verdicts. Out: the winner and what remains wrong.
G	Set gate	Builds contact sheets of crops beside each reference; flags and re-renders outliers.	In: accepted items with bounding boxes. Out: flagged items.
H	Orchestrator	Runs A and B, then items in parallel, then G. Owns retries, markers and resume.	One run record per set; one marker per image.

What your models must be able to do

MUST HAVE	HELPS A LOT
An image model that follows a reference image for identity, or a per-subject fine-tune	Several reference images per generation call
A vision model that accepts several images in one call	Bounding boxes from the vision model
Structured (JSON) output you can validate	An image embedding model, for outlier detection
Storage with a create-if-absent write	An image-edit call, for local repairs

DATA CONTRACTS

05 The five records that hold the system together

These are the shapes we converged on. Field names are ours; rename freely. What matters is which fields exist and which are free text.

WHY A SPEC AT ALL
The image tells the generator what to draw. The spec tells the judge what to check and gives repairs something exact to quote.

1. Subject reference and outfit spec

SUBJECT — built once per set

```
sheet:   image, content hash, likeness to the anchor
outfit:  { top, bottom, footwear, outerwear?, accessories[] }
slot:    { desc: free text incl. LENGTH words,
           seen: true | false,
           colours: ["#rrggbb", ...] }
text:    one sentence rendered from the slots  ← this is what is pinned
```

Read the spec off the model sheet, not off the original anchor: on the sheet every garment is visible. A slot the reader had to guess is a failed derivation, not a spec. Render the slots into one sentence and use that exact sentence in the prompt, in the judge's question and in repairs, so the three can never disagree.

2. Object reference

```
OBJECT                                                    one per distinct object

sheet:    image (two views, plain background), content hash
spec:     { kind, colours[], material, size relative to subject, marks[] }
contract: { kind: single | group | assembly | scene }   ← what the reference
depicts
text:     one sentence rendered from the spec
```

3. Appearance plan

```
PLAN                                                          one per set

object:     { id, name, aliases[], critical, design, instances[] }
occurrence: { item, instanceIds[], count: single | group,
              visibility: visible | absent | off_screen | optional,
              state: free text, evidence: exact quote from the script }
```

Two things here took us longest to learn. First, **a mention is not a presence**: a script that says "she remembered the map" does not mean the map is in the picture, so visibility is decided explicitly per item. Second, **identity is separate from state**: a broken or moved object keeps its id and its design; only its state changes.

4. Verdict

```
VERDICT                                                  one per candidate

subject:  same_subject, hair_match, skin_match, bbox
outfit:   per slot → match | mismatch | not_visible
objects:  per object → presence, look, duplicated, state_match, bbox
others:   per recurring character → present, look_match, duplicated, bbox
body:     extra_limbs, body_truncated, impossible_pose
status:   verified | unavailable
```

5. Marker

```
MARKER                                          stored beside every image

image hash · judge version · kit hash · plan hash ·
verdict · blocking[] · advisory[] · shipped_unresolved
```

A cached image is trusted only if its marker matches the current image bytes, judge version, kit and plan. Otherwise it is re-checked. Without this, a better judge never re-examines old work.

> **KEYS**
> Hash every input that can change pixels (kit, plan, script, style version) into the image's storage key. Unchanged inputs then replay for free and changed inputs re-render automatically, with no invalidation logic to maintain.

06

PROMPT PATTERNS

Wording that survived twenty revisions

These are patterns, lightly generalised from prompts we run in production. Adapt the nouns; keep the structure.

The model sheet

PATTERN · SUBJECT MODEL SHEET — one wide image

three FULL-BODY figures of the SAME <subject> standing side by side in one row — LEFT: front view, MIDDLE: three-quarter view, RIGHT: back view — relaxed, arms at the sides, head to toe entirely inside the frame, feet and shoes fully visible on every figure.

two small head-and-shoulders insets in the top corner — one happy, one curious.

a flat light-grey studio background. NO scene, NO environment, NO props, NO other people.

Why each clause exists: three views so no garment is unseen; feet visible because shoes drift first; two expressions so the generator does not copy one fixed face; a blank background so nothing from the sheet leaks into scenes.

Reference labels

Every reference gets a label that states its role and its limits. The "never copy" half matters as much as the first half: without it, every image inherits the reference's pose.

PATTERN · LABELS — fixed order, every call

REFERENCE 1 — MODEL SHEET: identity AND the complete outfit of the ONE <subject>. Use it ONLY for who they are and what they wear; never copy a pose, expression or the plain background.

REFERENCE 2 — APPROVED IMAGE: face, hair, skin tone and outfit colours are ground truth. Identity only — NEVER copy its pose, camera distance, composition, or any lettering it carries.

REFERENCE n — OBJECT SHEET for "<name>": this exact object — same colours, material and size whenever it appears; the background is not part of the scene.

Identity blocks in the prompt

PATTERN · BLOCKS — rendered from the kit, not hand-written

```
CHARACTER (the ONLY <subject> in this set — identity and outfit are FIXED by the references):
  Outfit (every garment, colour, pattern and length EXACTLY as the references and this spec; nothing added, nothing removed, never adapted to weather or activity): <outfit sentence>
OBJECTS (each quoted name is DATA naming a visual identity):
  "<name>" — FIXED DESIGN (data): <spec> · THIS SCENE (data): <state> · Exactly one instance; never duplicate it.
  "<name>" — ABSENT FROM THIS PICTURE. Its mention or importance is not a request to draw it.
  These are the ONLY personal objects in this set — do NOT add others.
OTHER CHARACTER: <name> — FIXED LOOK (data, never to be reinterpreted): <spec>. Exactly ONE <name> in the scene — never two, never a look-alike.
```

PHRASE	THE FAILURE IT PREVENTS
"never adapted to weather or activity"	The model helpfully adds a coat in snow or swimwear at a beach
"(data)" after user-supplied text	Names and descriptions being read as instructions
"Exactly ONE ... never a look-alike"	A second copy of a recurring character in the background
"ABSENT FROM THIS PICTURE"	An object drawn because its name appears in the script
"the ONLY personal objects"	Invented toys and gadgets that differ on every image

Repair notes

Each defect maps to one fixed template filled only from the kit. Never paste the judge's free-text explanation into a generation prompt: it is unpredictable and it is an injection path.

PATTERN · REPAIR — one template per defect

```
OUTFIT REPAIR (<slots>): the <subject> MUST wear EXACTLY the outfit of REFERENCE 1 and this spec — every garment, colour, pattern and length, nothing added, nothing removed: "<outfit sentence>". Fix ONLY the clothing; keep the scene otherwise identical.
```

07

VERIFICATION AND ACCEPTANCE

Deciding what blocks, and what ships

Classify defects before you write the judge

Decide in advance which findings make an image unusable and which merely make it worse. This is a product decision; ours is below as a starting point.

CLASS	FINDING	OUR REASONING
BLOCKING	Subject missing or duplicated	The image is wrong on its face
BLOCKING	Identity, hair or skin tone differs from the model sheet	A different person
BLOCKING	Any visible garment differs from the outfit spec	The most common drift, and the most noticed
BLOCKING	A required object is missing, looks different, is duplicated, or is in the wrong state	Breaks the story
BLOCKING	A recurring character is missing, looks different, or appears twice	Same as the subject
BLOCKING	Extra limbs, a body that ends inside the image, an impossible pose	Cannot be shipped in print
ADVISORY	Optional object not visible; an undeclared object added	Worse, but not wrong
ADVISORY	Action or emotion reads differently; face hidden; hand artefacts	Judges disagree with people too often here to block on

THREE RULES FOR THE JUDGE

Unavailable is not a pass. If the judge errors or returns a malformed answer, mark the candidate unchecked; rank it below every checked one and re-check it later. **Confirm the uncertain.** Where the judge is unreliable (for us, optional garments), require a second independent call before blocking. **Exempt the legitimate.** Write exceptions into the question (for us: bath scenes, lettering that is part of a garment).

The acceptance algorithm

PER ITEM — pseudocode

```
budget = 3                                  # total renders this item may buy, across every gate
candidates = render(n_first)                # each candidate keeps its own stored bytes
for c in candidates: c.verdict = judge(c, kit); c.score = score(c)
best = top(candidates)                      # rank by TIER first, then score

while best has blocking defects and budget remains:
    note = fixed_template(best.blocking, kit)       # never the judge's prose
    r = render(scene + note); r.verdict = judge(r, kit)
    if r is unchecked: break                        # never adopt what you could not verify
    if rank(r) >= rank(best): best = r

promote(best)                               # copy to the canonical key; write the marker
```

TIER	CANDIDATE
0	Checked, nothing blocking
1	Checked, something blocking
2	Unchecked

Rank by tier before score. Scores accumulate bonuses, and a bonus must never lift a blocking candidate over a clean one. Compare the tier first; use the score only inside a tier.

PER SET — pseudocode

```
for each reference R in kit (subject, each object, each recurring character):
    tiles = [R] + [crop(item, bbox_of_R_in_item) for item in set where R was expected]
    flagged = judge(contact_sheet(tiles), "which tiles differ from the first?")
    for item in flagged, only if rendered in THIS run and budget remains:
        r = rerender(item, cite R's index in the pack)
        keep r only if it is not worse than the original     # else restore the original bytes
```

Starting values

PARAMETER	WE USE	RECALIBRATE BY
Model-sheet candidates; first round in parallel	3; 2	Measuring first-candidate pass rate on your model
Minimum likeness of sheet to the approved anchor	0.8 (judged 0–1)	Comparing judge scores with human pass / fail on 50 sheets
Candidates per item; repair passes; total budget	1; 1; 3	Your cost ceiling and first-pass yield
Garment colour tolerance (CIE76 ΔE, band vs spec)	28	Your art style; painterly styles need more slack
Embedding outlier rule	> 2 SD, 6+ items	Plotting distances on sets you know are good and bad
Set-gate re-renders per run	3	How often the set gate is right
Contact-sheet tile size; columns	384 px; 4	The smallest detail you need legible

ADAPTING IT

08 How the method bends to different products

We have run this on one product. The table is our reasoning about others, not field experience; treat it as a set of hypotheses to test in your pilot.

PRODUCT	THE SET	WHAT CHANGES	WATCH FOR
Storybooks, comics	Pages or panels of one story	Closest to ours. Comics add several characters per panel and speech balloons.	Duplicated characters; which character is which
Brand or mascot content	A campaign's images	The kit is long-lived and human-approved once. Build it by hand; verify every output against it.	Logo and colour accuracy matter more than pose variety
Product imagery	One product in many scenes	The "subject" is an object. The object sheet and its spec carry everything.	Proportions, label text, material finish
Storyboards, video keyframes	Shots of one sequence	Add continuity of state between adjacent shots to the appearance plan.	State changes (wet, torn, open) must persist
Avatars, game characters	One character in many poses or outfits	Split identity from outfit: one face sheet, several outfit specs.	Outfit bleeding between variants

What to keep whatever the product

Keep
A purpose-made reference per identity; one spec shared by prompt and check; structured verdicts; blocking versus advisory; tier-first ranking; a set-level check; markers and hashed keys.

Tune
Which defects block; every threshold; how many candidates; whether to stop or ship when the budget is spent; how much variety you force between items.

Drop if it does not apply
The outfit spec when clothing may change by design; the appearance plan when there are no recurring objects; the photo-likeness signal when the subject is fictional.

Do not forget variety

Strong identity references pull every image toward the same pose and framing. Once consistency works, sets start to look like twelve copies. We fixed this with a deterministic plan that assigns each item a shot type and staging, so that "who" and "how it is framed" are pinned by two separate mechanisms that do not fight. Plan for this from the start; it is easier than retrofitting.

If the subject is a real person

- **Anchor on an approved generated image, not the photo.** A person approves one rendering; that rendering defines the identity. The photo then informs, as an advisory likeness score, but does not redesign anything.
- **Expect provider safety refusals.** Our judge call was refused when a real child's photo sat beside a full-body sheet with "compare the faces" instructions. Have a fallback that runs the check without the photo, and a prompt ladder for generation that steps down to more neutral wording.
- **Keep the photo out of generation calls** wherever you can. Fewer copies of personal data in fewer requests is simpler to defend.

BUILD ORDER

09 Six phases, each with a way to know it worked

The order matters. Each phase is useful alone, and each gives the next one something to stand on. Start by measuring, or you will not be able to tell which phase helped.

1. **Phase 0 · Baseline**
   Assemble 20 to 30 real sets from your current pipeline. Have two people mark every inconsistency by type.
   Exit: a table of defect rates by type, and agreement between your two reviewers.

2. **Phase 1 · Identity kit and reference pack**
   Build the model sheet and outfit spec (by hand at first, if that is faster). Attach the labelled pack to every item. No checks yet.
   Exit: the outfit and identity defect rates fall on the same 20 to 30 sets. This phase usually gives the largest single gain.

3. **Phase 2 · Structured judge**
   Write the verdict schema and the defect ledger. Run the judge on the Phase 0 sets and compare with your human marks.
   Exit: you know the judge's precision and recall per defect, and you have moved the unreliable ones to advisory.

4. **Phase 3 · Selection and repair**
   Add candidates, tier-first ranking, repair templates and the per-item budget.
   Exit: blocking defects per accepted image fall, and renders per accepted image stay under your cost ceiling.

5. **Phase 4 · Set gate**
   Add contact sheets for the subject, then for objects and other characters.
   Exit: sets that pass every per-item check but still look inconsistent to a person become rare.

6. **Phase 5 · Objects and operations**
   Add the appearance plan with explicit visibility, automated kit election, markers, hashed keys, resume, and recovery paths for references that fail their own check.
   Exit: a failed run resumes without repaying for finished work, and no single failed reference can stall a set for ever.

What to measure throughout

METRIC	DEFINITION	WHY
Set consistency rate	Share of sets a human reviewer accepts with no identity or object defect	The number the business cares about
Defect rate by type	Blocking findings per 100 accepted images, per ledger row	Shows which mechanism to work on next
First-pass yield	Share of items whose first candidate has nothing blocking	Measures the fixed inputs, separately from the gates

Renders per accepted image	All generation calls divided by images shipped	Your cost, and an early warning of a prompt regression
Judge agreement	Precision and recall of each judge field against human marks	A gate is only as good as its judge
Unchecked rate	Share of candidates whose verdict was unavailable	Silent judge outages look like quality gains otherwise

Cost model, in calls

We give call counts, not prices, because prices are yours. With our defaults, for a set of N items with K object references:

STAGE	GENERATION CALLS	JUDGE / TEXT CALLS
Subject model sheet	2 to 3	one per candidate
Outfit spec	0	1
Object references	1 to 3 per object	one per candidate
Appearance plan	0	2 to 3 text calls
Items	N, up to 3N in the worst case	one per candidate
Set gate	up to a few re-renders	about 2 + K

The kit is paid once per set and reused on every retry. In steady state, the verification layer roughly doubles the number of model calls and adds well under one extra generation per item.

PITFALLS

10 Twelve mistakes we made so you do not have to

INPUTS

1. Fixing drift with more prompt text

We spent four revisions adding sentences. Repeating the outfit six times added conflicting phrasings, not weight. Replace words with a picture.

INPUTS

2. Deriving the spec from the wrong image

A spec read off a cropped hero image has to guess the shoes. Read it off the model sheet, where everything is visible.

INPUTS

3. Chaining the previous image

Drift compounds, compositions converge. Fixed references only.

INPUTS

4. A safety rule that silently removed a feature

"No other humans in the background" meant a recurring adult character was never drawn consistently. Make exceptions explicit and bounded.

CHECKS

5. A gate without a better input

A gate can only reject. If the first render is usually wrong, a gate just burns budget. Pair every check with a pinned input that makes the right result likely.

CHECKS

6. Judging at thumbnail scale

A shoe is invisible in a grid of twelve whole images. Crop the subject and compare crops.

CHECKS

7. Treating "judge unavailable" as a pass

An outage then looks like a quality improvement. Track unchecked as its own state.

CHECKS

8. Token limits clipping the verdict

Reasoning tokens counted against the output limit once truncated every verdict and failed every set. Give judges generous output limits and name the finish reason on a parse failure.

MATCHING

9. Substring matching on names

A world called "Farm" masked the first four letters of "Farmer". Match names as whole words, and test with names that contain each other.

MATCHING

10. Assuming a mention means presence

Objects were drawn because the script named them. Decide visibility explicitly.

OPERATIONS

11. A reference that fails and replays for ever

Content-addressed caching will faithfully replay a saved rejection. Give every reference an ordered way out: second opinion, re-plan, degrade to text, flag.

OPERATIONS

12. Repairs that overwrite better work

Store every candidate under its own key. Adopt a repair only if it ranks at least as high, and restore the original if a set-gate re-render comes back worse.

THE PATTERN BEHIND THE LIST

Almost every item is the same mistake in a different place: an invariant that was stated but not pinned, or pinned but not checked, or checked but with no path to recovery. When something drifts, ask which of the three is missing.

11

THE HANDOFF

How we propose to work together

This chapter is a proposal. Dates, people and scope are for both teams to agree.

What we hand over

DELIVERABLE	CONTENTS	FORM
This playbook	Method, contracts, patterns, algorithm, pitfalls	PDF
Prompt pack	The full text of our sheet, label, block, judge and repair prompts, with the subject generalised	Text files
Schema pack	The five records as JSON Schema, with example instances	JSON
Defect ledger	Every defect string, its class, its repair template	Spreadsheet
Evaluation kit	The review rubric and scoring sheet for the Phase 0 baseline	Spreadsheet
Working sessions	Four sessions, below	Calls

What we do not hand over

- **Customer data.** No photos, names, books or generated images of real customers. Examples will be synthetic or made for the purpose.
- **Credentials and infrastructure.** No keys, buckets or access to our systems.
- **Source code**, unless both companies agree otherwise in writing. The playbook is designed to stand without it.

Sessions

1. **Kickoff · 60 minutes**
   You walk us through your product and your answers to chapter 03. We agree which parts of the method apply and what "consistent" means for you.
   Output: a one-page scope and the list of identities to pin.

2. **Design review · 90 minutes**
   You present your mapping of the eight components onto your stack. We review the contracts and the defect ledger you drafted.
   Output: an agreed architecture and a blocking / advisory list.

3. **Pilot review · 90 minutes, after Phases 0 to 2**
   We look at your baseline numbers, your model sheets and your judge agreement together.
   Output: thresholds for your data and a go / no-go on Phases 3 to 5.

4. **Close-out · 60 minutes, after Phase 4**
   Before-and-after numbers, open problems, and anything you learned that we should fold back into this playbook.
   Output: a short written summary both teams sign off.

Who we need from you

- A product owner who can say what blocks
- An engineer who owns the generation pipeline
- Someone who writes and tests prompts
- Two reviewers for the baseline

Success, stated up front

- A measured baseline exists before any change
- Set consistency rate improves on the same sets
- Renders per accepted image is within your ceiling
- Your team can add a new identity type unaided

Risks to agree on early

RISK	HOW WE WOULD HANDLE IT
Your image model follows references weakly	Find out in week one with a ten-image test: one model sheet, ten scenes. If it fails, the conversation is about the model, not the method.
Your vision model is an unreliable judge	Phase 2 measures this directly. Unreliable fields become advisory; reliable ones gate.
Cost per set is too high	Drop to one candidate and one repair, keep the kit and the set gate. Most of the gain is in the fixed inputs.
Our thresholds do not fit your style	Expected. Chapter 07 lists how to recalibrate each one.

APPENDIX

A Readiness checklist

Tick these off as you go. A set-generation system that can answer yes to all of them has the whole method in place.

Fixed inputs

- [ ] Every identity that must persist has a purpose-made reference image
- [ ] Each reference has a structured spec read off that reference
- [ ] References are built before any item and never rebuilt mid-set
- [ ] A reference is stored once; concurrent builders adopt one winner
- [ ] An unverified reference is never adopted
- [ ] Every item gets the same references in the same order with the same labels
- [ ] Labels say what not to copy
- [ ] No item receives another item as a generation reference
- [ ] Presence of each object in each item is decided explicitly

Verification

- [ ] The judge sees the references beside the image
- [ ] The verdict is a validated schema
- [ ] Every finding maps to a fixed string, classed blocking or advisory
- [ ] A malformed or missing verdict is "unchecked", not "pass"
- [ ] Judge agreement with humans is measured per field

Acceptance

- [ ] Candidates are ranked by tier before score
- [ ] Every candidate keeps its own stored bytes
- [ ] Repair prompts come from templates and kit data only
- [ ] An unverified repair is never adopted
- [ ] There is one render budget per item across all gates
- [ ] The finished set is checked as a set, on crops
- [ ] A set-gate re-render that is worse is discarded
- [ ] The policy when budget runs out is written down

Operations

- [ ] Storage keys hash every input that changes pixels
- [ ] A marker beside each image records what was checked, by which judge version
- [ ] A stale marker causes a re-check
- [ ] A failed run resumes without repaying for finished work
- [ ] A reference that fails its own check has an ordered recovery path
- [ ] Findings that shipped are visible to whoever reviews the set